



MORE ON PERCEPTRON LEARNING

INT301 Bio-computation, Week 3, 2025



Outline

梯度下降法则

■ Gradient Descent Rule

先找点 (x_1, x_2) ，然后根据每个点的输入与权重和 $w_1x_1 + w_2x_2 = z$ ，并算出偏置项 b ，再用 z 与阈值 r 对比，找出对应的 t 标签

逻辑回归：当不能完全分割时，找其中最好的 w 组合。逻辑回归可以充当评判标准

将原来 $w^T x$ 输出的固定结果 z 转变为 $G(z) = \frac{1}{1+e^{-z}}$ ，然后用 $G(z)$ 充当概率

$y = G(z)$ 代表真实标签概率， $1-y$ 代表错误标签

此时的损失函数 $L(y, t) = -t \log y - (1-t) \log (1-y)$ 就能平滑地给出正确的梯度变化。

The Gradient Descent Rule

感知机的目的就是
就是为了找到
能完美分开所有
数据点的决策
边界, 如果完美
边界不存在, 那么
感知机就不会
停止.

- Perceptron rule fails **if data is not linearly separable**

如果数据不是线性可分的, 那么感知机会失效

- **Idea:** uses **gradient descent** to search the **hypothesis space**

用梯度下降来寻找假设空间

- perceptron rule cannot be used (not differentiable)
- hence, an **unthresholded linear unit** is an appropriate

为了能够沿着梯度向下直到 critical point,
我们需要确保两样东西: 一个是能衡量
错误程度的方法 (梯度越低, 错误越小).
一个是计算坡度的方法. (确保一直向下).

$\sum_e$: 对所有 error measure: 均方差公式

训练样本进行计算后求和 (正确答案)

y_e 是第 e 个样本的真实标签

o_e 是模型对第 e 个样本的输出值

$$E(w) = \frac{1}{2} \sum_e (y_e - o_e)^2$$

- To understand gradient descent, it is helpful to visualize the entire hypothesis space with

- all possible weight vectors

$(y_e - o_e)^2$ 计算了单个样本上, 真实值与预测值差距的平方
平方确保是正数, 并且会放大较错的错误

- associated E values

需要 unthresholded linear unit 的原因是, 原始感知机输出是 0 或 -1,
但是这样会有平滑过度无法计算出坡度, 因此公式中 o_e 是未经激活函数处理的原始加权和 $\sum w_i x_i$, 是平滑的.

但是这样会有平滑过度无法计算出坡度, 因此公式中 o_e 是未经激活函数处理的原始加权和 $\sum w_i x_i$, 是平滑的.

The Gradient Descent Rule

- The objective is to minimize the following error:

$$E(w) = \frac{1}{2} \sum_e (y_e - o_e)^2$$

总误差

为找到一组最佳的权重 w .

- The training is a process of minimizing the error $E(w)$ in the steepest direction (most rapid decrease), **that is in direction opposite to the gradient:**

$\nabla E(w)$ 是一个包含了所有权重偏导数的向量, $\nabla E(w)$ 这个向量指向的是函数值(总误差)增长最快的方向.

$$\nabla E(w) = [\partial E / \partial w_0, \partial E / \partial w_1, \dots, \partial E / \partial w_d]$$

梯度

which leads to the **gradient descent training rule:**

权重更新法则

梯度向量的正相反方向进行调整, 误差减小, 最快方向.

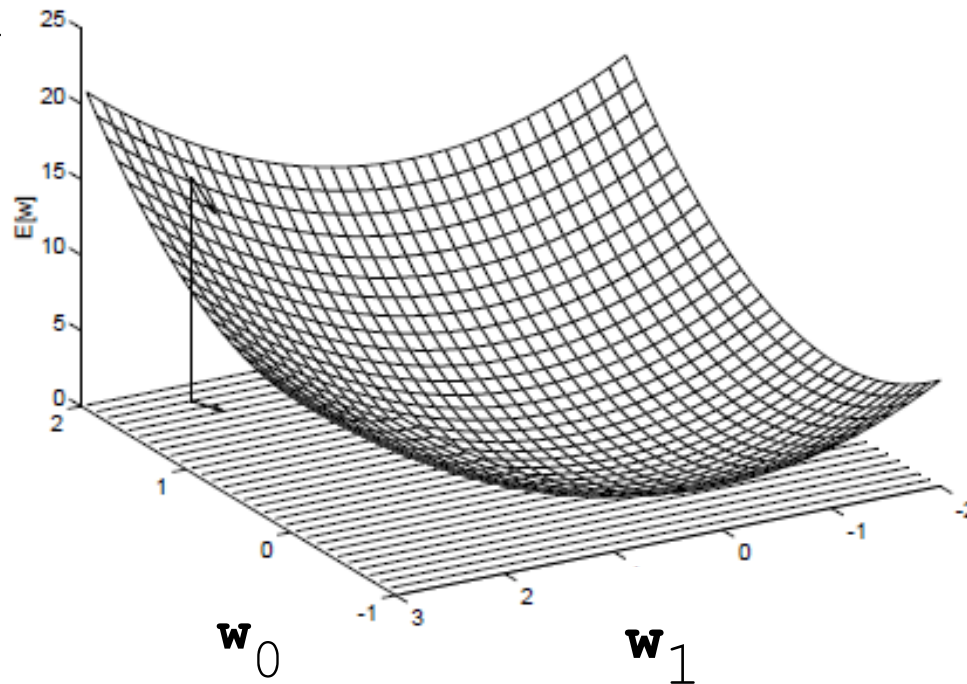
$$w_i = w_i - \eta \partial E / \partial w_i$$

Error Surface

In this module, we assume all the error surface is convex.

- the axes w_0 , w_1 represent possible values for the two weights of a simple linear unit

$\Delta E(w)$: 对于每一组给定的 (w_0, w_1) 组合, 我们模型的总误差是多少



error surface must be **parabolic** with a **single global minimum**

Moving Downhill: Move in direction of negative derivative

导数为正: 表示函数图像在这一点是向上倾斜 (从左向右看)。

导数为负: 表示函数图像在这一点是向下倾斜

导数为零: 意味该位置是 critical point.

Decreasing $E(\underline{w})$

此时 $E(w)$ 是正数

$$\frac{dE(w)}{dw_1} > 0$$

$$w_i = w_i - \eta \frac{dE(w)}{dw_i}$$

会让 w_i 变小

当 $E(w)$ 位于最低点右侧, 曲线是向下倾斜向左, 为减小误差

$E(w)$, 我们需要减小 w_1 .

$\frac{dE(\underline{w})}{dw_1}$
斜率

$$\frac{dE(\underline{w})}{dw_1} > 0 \quad w_1 \leq w_1 - \eta \frac{dE(\underline{w})}{dw_1}$$

i.e., **the rule decreases w_1**

Moving Downhill: Move in direction of negative derivative



$\frac{dE(w_1)}{dw_1}$ 是一个负数

$w_L = w_L - \eta \frac{dE(w_L)}{dw_L}$
会让 w_L 变大

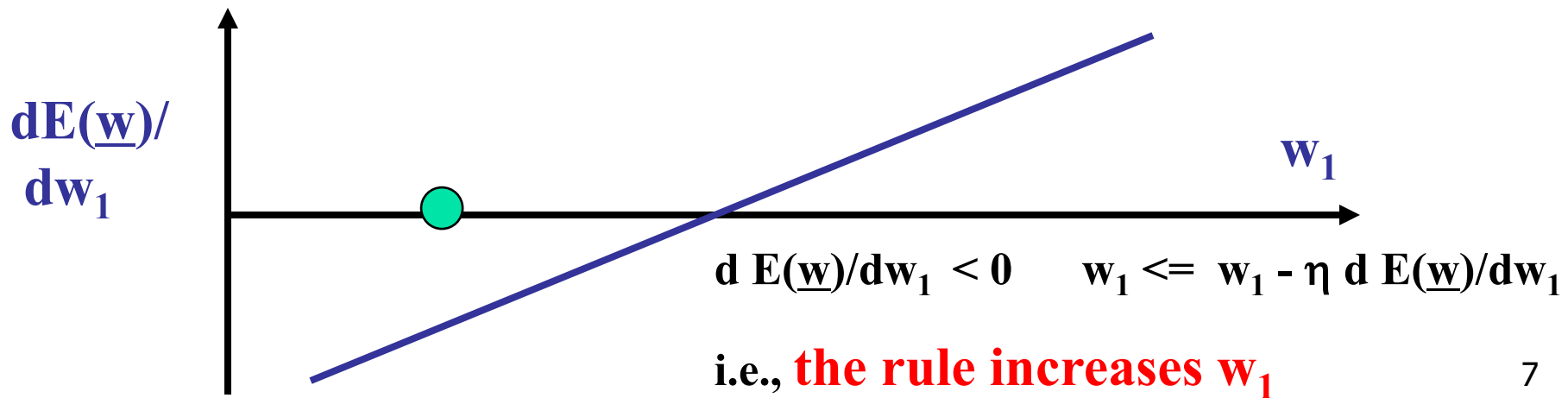
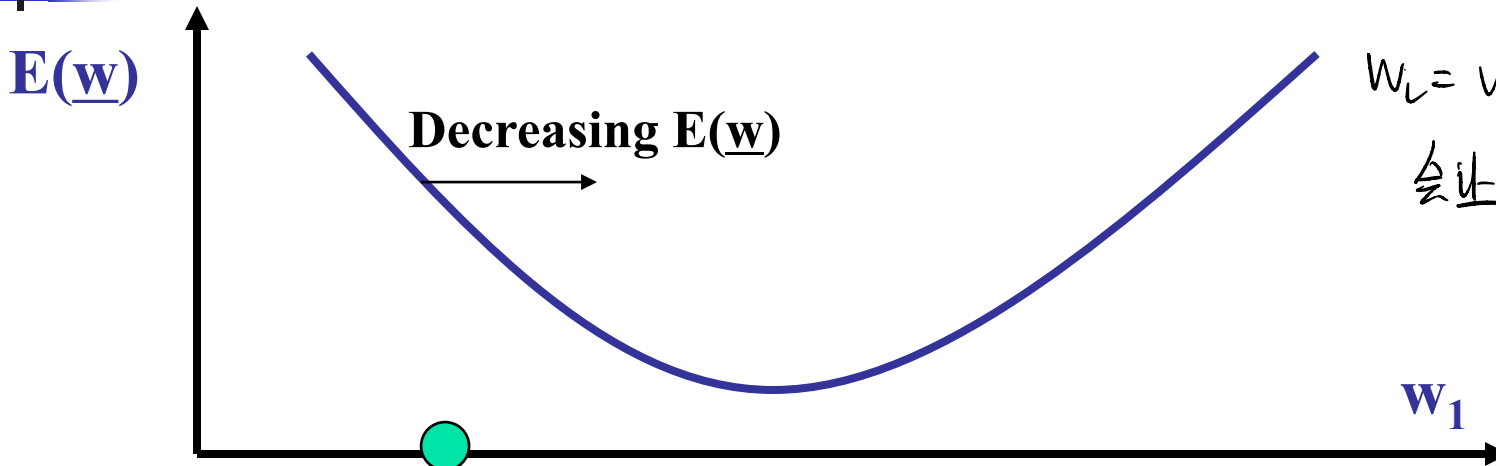


Illustration of Gradient Descent

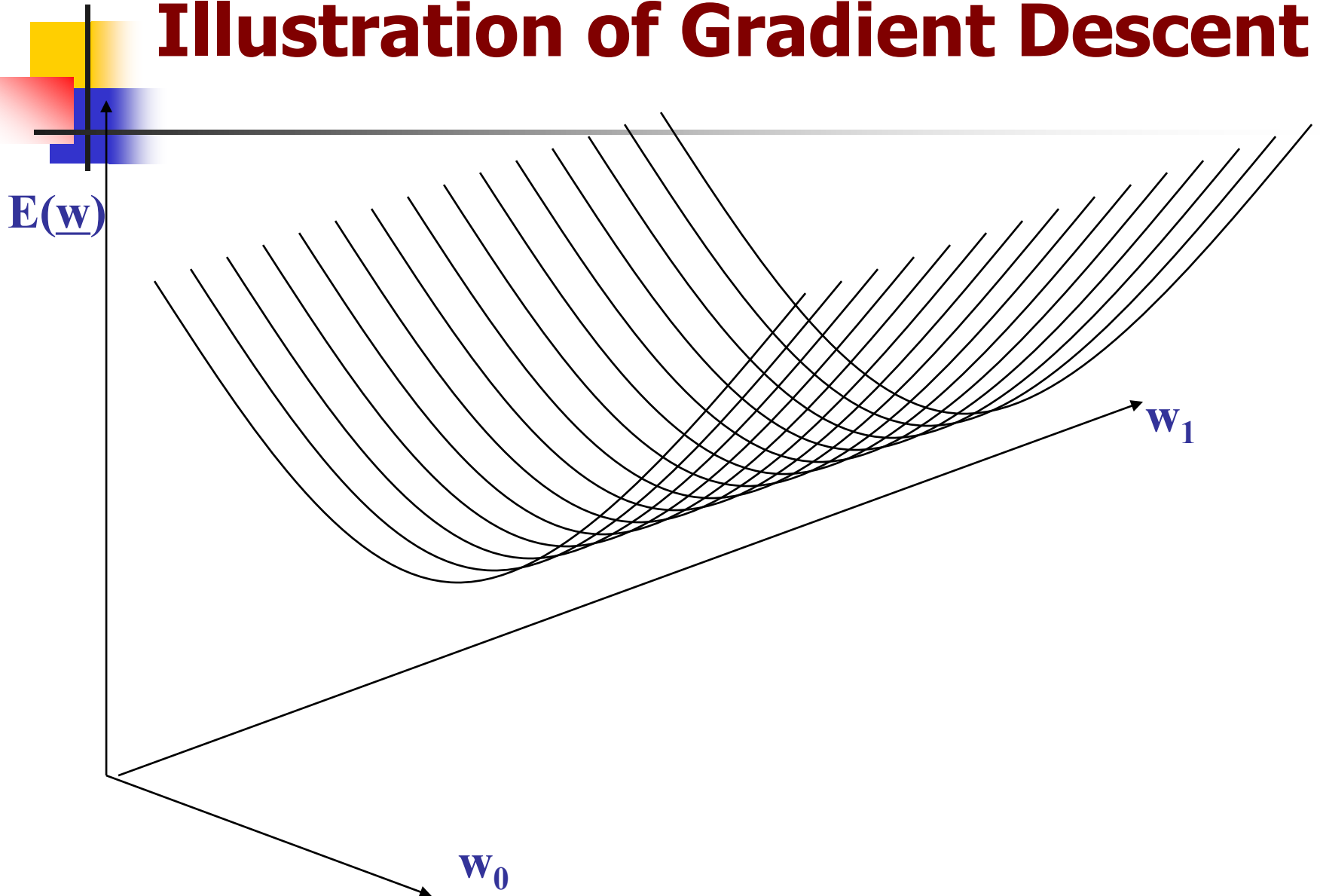


Illustration of Gradient Descent

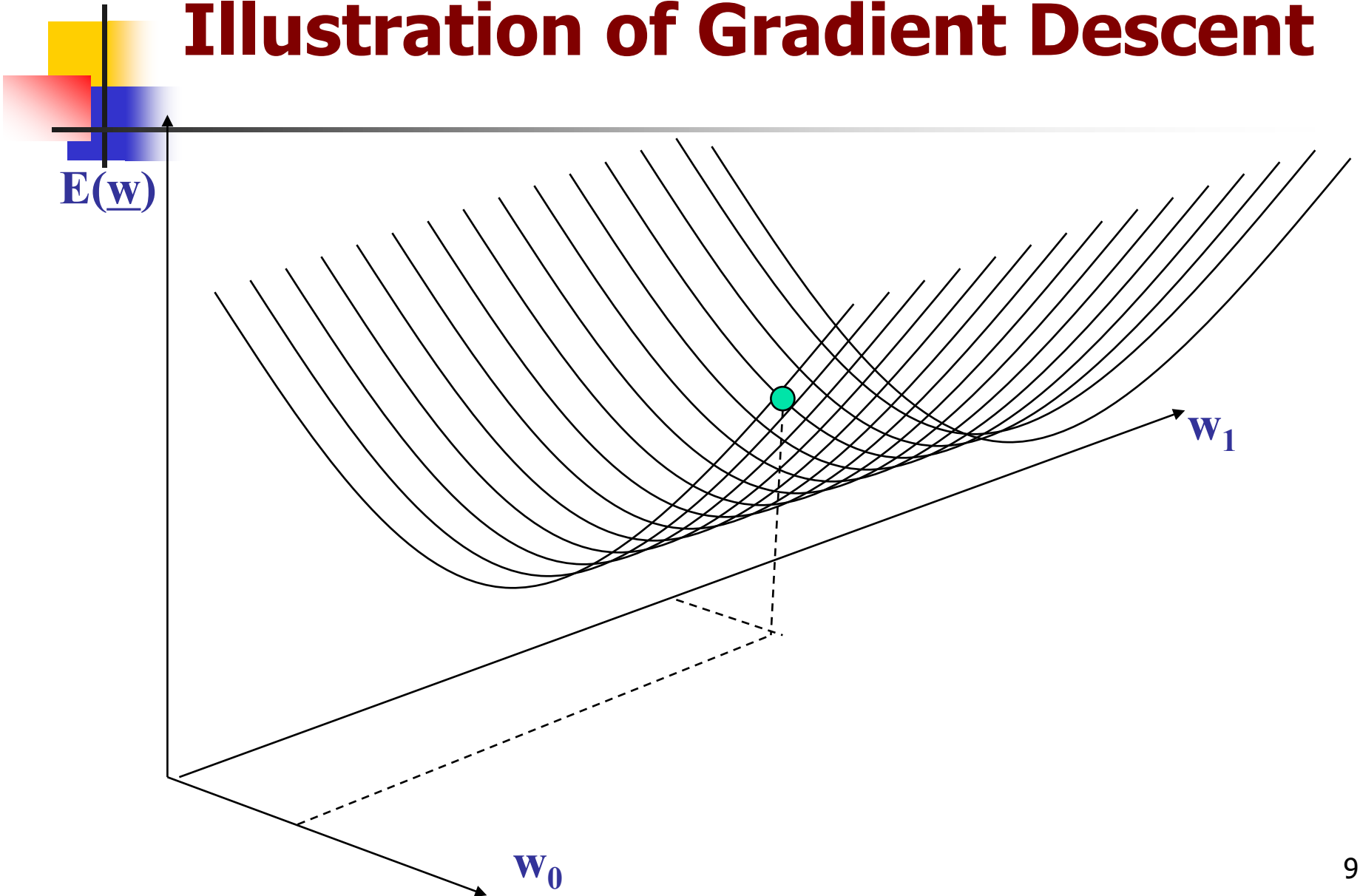


Illustration of Gradient Descent

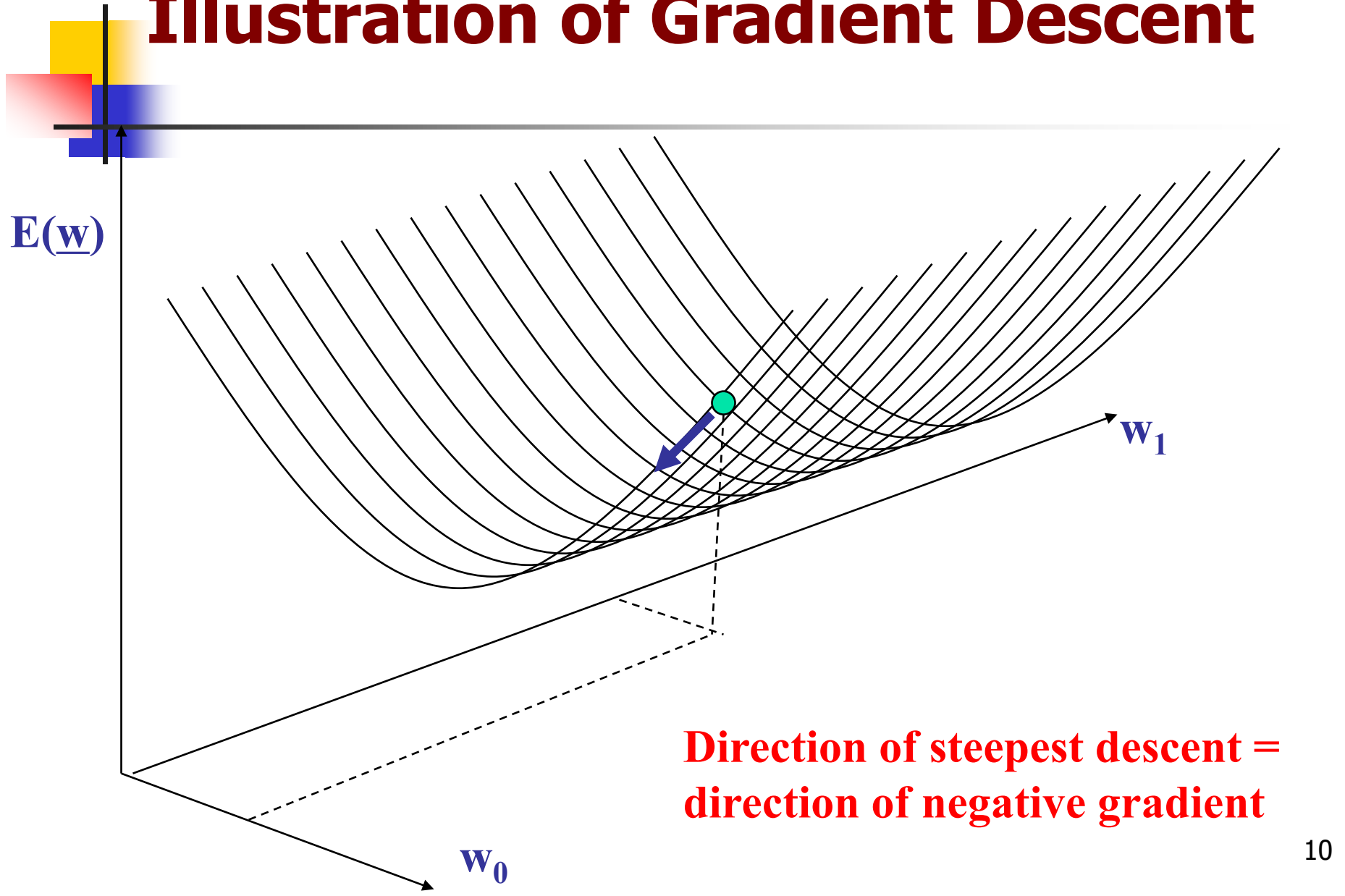
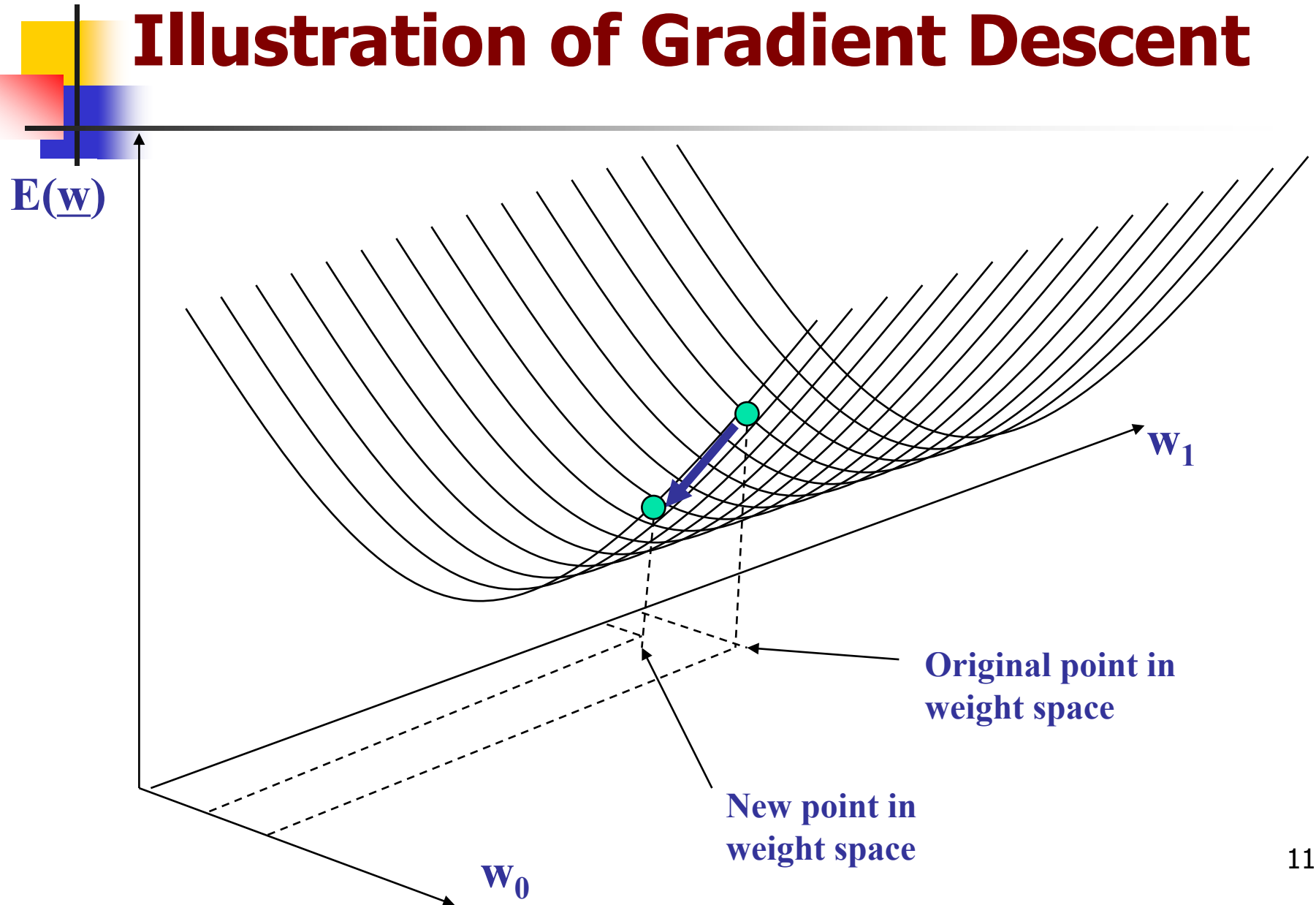


Illustration of Gradient Descent



The Gradient Descent Rule

- The weight update can be derived as follows:

$$\begin{aligned}
 \frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \left(\frac{1}{2} \sum_e (y_e - o_e)^2 \right) & \frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \left(\frac{1}{2} \sum_e (y_e - o_e)^2 \right) \\
 &= \frac{1}{2} \sum_e \frac{\partial (y_e - o_e)^2}{\partial w_i} & &= \frac{1}{2} \sum_e \frac{\partial}{\partial w_i} (y_e - o_e)^2 \\
 &= \frac{1}{2} \sum_e 2(y_e - o_e) \frac{\partial (y_e - o_e)}{\partial w_i} & &= \frac{1}{2} \sum_e 2(y_e - o_e) \frac{\partial (y_e - w_i x_{ie})}{\partial w_i} \\
 &= \sum_e (y_e - o_e) \frac{\partial (y_e - w_i x_{ie})}{\partial w_i} & &= \sum_e (y_e - o_e) (-x_{ie}) \\
 &= \sum_e (y_e - o_e)(-x_{ie})
 \end{aligned}$$

where x_{ie} denotes the i -th component of the example e .

The gradient descent training rule becomes:

梯度下降训练法则

$$w_i = w_i + \eta \sum_e (y_e - o_e) x_{ie}$$

替换 $\frac{\partial E}{\partial w}$

梯度下降的完整算法流程

Gradient Descent Learning Algorithm

算法开始前, 需要准备好训练数据 (x_e, y_e) . 随机设置一组初始的权重 w_i , 确定学习率 η .

- **Initialization:** Examples $\{(x_e, y_e)\}_{e=1}^N$, initial weights w_i set to small random values, learning rate parameter η

- **Repeat** 重复训练, 直到满足误差足够小或达到最大迭代次数.

for each training example (x_e, y_e)

- calculate the network output: $o_e = \sum_{i=0}^d w_i x_{ie}$ 使用当前权重, 计算模型预测值 $o_e = \sum w_i x_{ie}$

- if the Perceptron does not respond correctly, 计算权重校正量:
compute weight corrections: 计算这个样本产生的权重校正量并累加到总的校正量上.

$$\Delta w_i = \Delta w_i + \eta(y_e - o_e)x_{ie}$$

update the weights with the **accumulated error** from all examples

$$w_i = w_i + \Delta w_i$$

Gradient
Descent Rule

until termination condition is satisfied. 在遍历完所有训练样本之后, 用累加得到的总校正量 Δw_i 来更新权重. 13

Example

开始前, 这校正量 $\Delta W_1, \Delta W_2, \Delta W_0$ 初始化为 0.

- Suppose an example of Perceptron which accepts two inputs x_1 and x_2 , with weights $w_1 = 0.5$ and $w_2 = 0.3$ and $w_0 = -1$, learning rate = 1.
- Let the example is given: $x_1 = 2, x_2 = 1, y = 0$
The network output of the Perceptron is :

$$o = 2 * 0.5 + 1 * 0.3 - 1 = 0.3$$

- The weight updates according to the **gradient descent algorithm** will be:

$$\Delta W_1 = (0 - 0.3) * 2 = -0.6 \quad 0 + \eta (y - o) x_1$$

$$\Delta W_2 = (0 - 0.3) * 1 = -0.3 \quad 0 + \eta (y - o) x_2$$

$$\Delta W_0 = (0 - 0.3) * 1 = -0.3 \quad 0 + \eta (y - o) x_0$$

Example

- Let another example is given: $x_1 = 1, x_2 = 2, y = 1$
- The network output of the Perceptron is :
$$o = 1 * 0.5 + 2 * 0.3 - 1 = 0.1$$
 注意, 这里仍然使用原始权重.

The weight updates according to the gradient descent algorithm will be:

$$\Delta W_1 = -0.6 + (1 - 0.1) * 1 = 0.3$$

$$\Delta W_2 = -0.3 + (1 - 0.1) * 2 = 1.5$$

$$\Delta W_0 = -0.3 + (1 - 0.1) * 1 = 0.6$$

If there are no more examples, the weights will be modified as follows:

新权重 {

$$\begin{cases} W_1 = 0.5 + 0.3 = 0.8 \\ W_2 = 0.3 + 1.5 = 1.8 \\ W_0 = -1 + 0.6 = -0.4 \end{cases}$$

要在所有样本都训练完后, 才能更新权重.

Incremental gradient descent

批量梯度下降法存在两个主要问题：
①收敛非常慢：必须遍历所有训练样本，计算完累积的总误差之后，才能对权重进行一次更新。
②可能陷入局部最小值。

The gradient descent rule faces two difficulties in practice:

- it converges very slowly
- if there are multiple local minima in the error surface, then there is no guarantee that it will find the global minimum

增量梯度下降：计算一个样本的误差 → 更新一次权重 → 再取下一个样本。

- That is why, a **stochastic version** called **incremental gradient descent** rule is developed to overcome these difficulties.

Whereas the gradient descent rule updates the weights after calculating the whole error accumulated from all examples, the incremental version approximates the gradient descent error decrease by updating the weights after each training example.

Incremental gradient descent

- Incremental gradient descent is implemented 初始化

$$w_i = w_i + \eta(y_e - o_e)x_{ie} \quad \text{where} \quad o_e = \sum_{i=0}^d w_i x_{ie}$$

准备训练集. 随机初始化权重 w_i , 设过学习率 η

Incremental Gradient Descent Learning Algorithm

Initialization: *Examples $\{(x_e, y_e)\}_{e=1}^N$, initial weights w_i set to small random values, learning rate parameter η*

Repeat

for each training example (x_e, y_e)

在循环中,对每一个训练样本

(x_e, y_e) 执行以下操作:

- **calculate the network output:**

1. 计算网络输出 $o_e = \sum w_i x_{ie}$
2. 立即更新权重: 根据单个样本的误差,

$$o_e = \sum_{i=0}^d w_i x_{ie}$$

马上使用 Delta 法则更新权重:

$$w_i = w_i + \eta (y_e - o_e) x_{ie}$$

- **if the Perceptron does not respond correctly update the weights:**

$$w_i = w_i + \eta (y_e - o_e) x_{ie}$$

until termination condition is satisfied.

这样每次更新的方向是由单个样本决定的, 收敛更快, 且容易跳出局部最小值.

S型感知机

Sigmoidal Perceptrons

简单感知机的局限性: 带有
阈值或线性函数的简单单
层感知机无法被推广到像
多层神经网络这样更强大的
学习机致中

- The simple single-layer Perceptrons with threshold or linear activation functions are not generalizable to more powerful learning mechanisms like multilayer neural networks.

多个线性层堆叠在一起, 其最终效果等同于一个单独的线性层, 无法学习复杂的非线性关系

- That is why, single-layer Perceptrons with sigmoidal activation functions are developed.

阈值函数的局限性: 它的输出是跳变的 (直接从0变成1), 函数不平滑, 因此不可导

- The sigmoidal Perceptron produces output:

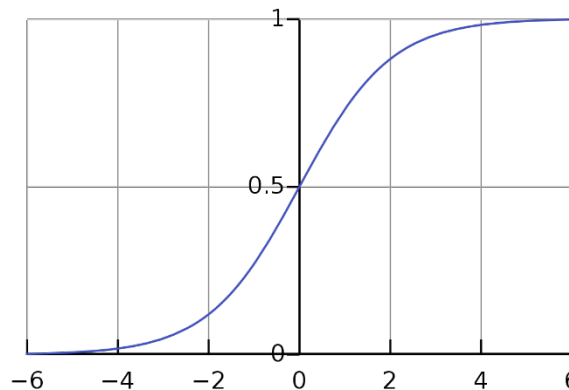
$$o = \sigma(S) = \frac{1}{1 + e^{-S}},$$

where:

无论S的值是大是小, 最终 $\sigma(S)$ 都能确定在(0,1)区间内

$$S = \sum_{i=0}^d w_i x_i$$

将原本单个样本的加权和 $\sum w_i x_i$ 转变为 $\sigma(S)$



Training Sigmoidal Perceptrons

- The gradient descent rule for training sigmoidal Perceptrons is again:

$$w_i = w_i - \eta \partial E / \partial w_i$$

梯度下降法则的公式不变, 但是 $\frac{\partial E}{\partial w_i}$ 因为 o_e 变成了 $\sigma(s)$, 所以细节会有变化.

- The difference is in the error derivative $\partial E / \partial w_i$ which due to the use of the sigmoidal function $\sigma(s)$ becomes:

$$\partial E / \partial w_i = \partial ((1/2) \sum_e (y_e - o_e)^2) / \partial w_i$$

$$= (1/2) \sum_e \partial (y_e - o_e)^2 / \partial w_i$$

$$\frac{\partial (y_e - o_e)^2}{\partial w_i} = 2(y_e - o_e) \frac{\partial (y_e - o_e)}{\partial w_i}$$

$$= (1/2) \sum_e 2(y_e - o_e) \partial (y_e - o_e) / \partial w_i$$

然后 o_e 替换为 $\sigma(s)$
 $\sigma(s)$ 的导数是 $\sigma'(s)$.

$$= \sum_e (y_e - o_e) \partial (y_e - \sigma(s)) / \partial w_i$$

$$= \sum_e (y_e - o_e) \sigma'(s) (-x_{ie})$$

where x_{ie} denotes the i -th component of the example

Training Sigmoidal Perceptrons

- The *Gradient descent training rule* for training sigmoidal Perceptrons is:

$$w_i = w_i + \eta \sum_e (y_e - o_e) \underline{\sigma'(S)} x_{ie}$$

where:

$$\sigma'(S) = \sigma(S)(1 - \sigma(S))$$

Gradient Descent Learning Algorithm for Sigmoidal Perceptrons

S型感知机的批量梯度下降训练过程

- *Initialization: Examples $\{(x_e, y_e)\}_{e=1}^N$, initial weights w_i set to small random values, learning rate parameter η* $s = \sum w_i x_i$
- *Repeat 对于每一个训练样本*
for each training example (x_e, y_e) 计算网络的输出 $o = \sigma(s)$ 其中 s 是输入的加权和
 - *calculate the network output: $o = \sigma(s)$ where $s = \sum_{i=0}^d w_i x_{ie}$*
 - *if the Perceptron does not respond correctly compute weight corrections: 计算该样本产生的权重校正量, 并将其累加到总校正量 Δw_i*

$$\Delta w_i = \Delta w_i + \eta(y_e - o_e)\sigma(s)(1 - \sigma(s))x_{ie}$$

注意 $\sigma(s)(1 - \sigma(s)) = 0(1 - 0)$
- update the weights with the accumulated error from all examples $w_i = w_i + \Delta w_i$ 在遍历完所有样本后, 用累加得到的总校正量 Δw_i 对权重进行一次总的更新.*

$$w_i = w_i + \Delta w_i$$
- until termination condition is satisfied.*

Example

- Suppose an example of Perceptron which accepts two inputs x_1 and x_2 , with weights $w_1 = 0.5$ and $w_2 = 0.3$ and $w_0 = -1$, learning rate = 1.

- Let the following example is given: $x_1 = 2, x_2 = 1, y = 0$

The output of the Perceptron is : 在批量梯度下降中, 每轮中, 每个样本都用统一的 w_i 训练, 累加完 Δw_i 之后下一轮再用新 w_i .

$$O = \sigma(-1 + 2 * \underline{0.5} + 1 * \underline{0.3}) = \sigma(0.3) = 0.5744$$

- The weight updates according to the gradient descent algorithm will be:

$$\Delta w_0 = (0 - 0.5744) * 0.5744 * (1 - 0.5744) * 1 = -0.1404$$

$$\Delta w_1 = (0 - 0.5744) * 0.5744 * (1 - 0.5744) * 2 = -0.2808$$

$$\Delta w_2 = (0 - 0.5744) * 0.5744 * (1 - 0.5744) * 1 = -0.1404$$

Example

Let another example is given: $x_1 = 1$, $x_2 = 2$, $y = 1$

The output of the Perceptron is :

$$O = \sigma(-1 + 1 * \underline{0.5} + 2 * \underline{0.3}) = \sigma(0.1) = 0.525$$

The weight updates according to the gradient descent algorithm will be:

$$\Delta w_0 = -0.1404 + (1 - 0.525) * 0.525 * (1 - 0.525) * 1 = -0.0219$$

$$\Delta w_1 = -0.2808 + (1 - 0.525) * 0.525 * (1 - 0.525) * 1 = -0.1623$$

$$\Delta w_2 = -0.1404 + (1 - 0.525) * 0.525 * (1 - 0.525) * 2 = 0.0966$$

If there are no more examples in the batch, the weights will be modified as follows:

$$w_0 = -1 + (-0.0219) = -1.0219$$

$$w_1 = 0.5 + (-0.1623) = 0.3966$$

$$w_2 = 0.3 + 0.0966 = 0.3966$$

Incremental Gradient Descent Learning

Algorithm for *Sigmoidal Perceptrons*

S型感知机的增量梯度下降学习算法。与上面的区别主要在于更新权重的时机。

Initialization:

Examples $\{(x_e, y_e)\}$, initial weights w_i set to small random values, learning rate parameter η

Repeat

计算网络输出: 首先计算加权和

for each training example (x_e, y_e)

$s = \sum w_i x_{ie}$ 然后将其通过 sigmoid

- calculate the network output: $o = \sigma(s)$ 函数得到输出 $o = G(s)$.

where $s = \sum_{i=0}^d w_i x_{ie}$

更新权重: 立即使用当前这一个样本的信息, 根据下面的法则更新网络中所有权重

- If the Perceptron does not respond correctly update the weights: $w_i = w_i + \eta(y_e - o_e)\sigma(s)(1 - \sigma(s))x_{ie}$

不再等其他权重累加

until termination condition is satisfied.

边学习, 边改进

注意: 普通感知机使用感知机学习法则, S型感知机使用梯度下降.

Perceptron vs. Gradient Descent

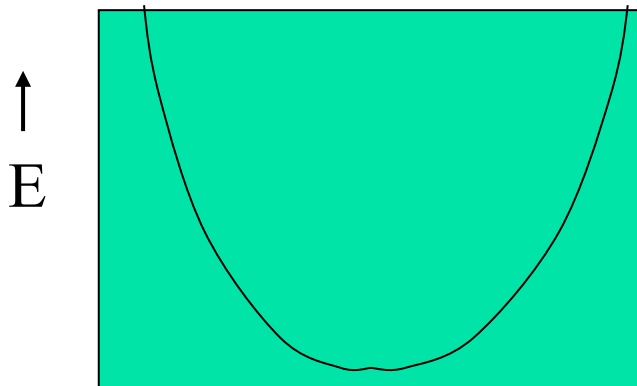
梯度下降找的是最小化均方误差, 即目标值与网络线性输出值之间的误差, 而不是和

- Gradient descent finds the decision boundary which minimizes the *sum squared error* of the (target - net) value rather than the (target - output) value output 的误差
感知机的目标是最小化分类错误。它会寻找一个能完美分类所有训练数据的决策边界, 只要是线性可分的。
- Perceptron rule will find the decision boundary which minimizes the classification error – *if the problem is linearly separable*
- Gradient descent decision boundary may leave more instances misclassified as compared to the perceptron rule: *could have a higher misclassification rate than with the perceptron rule* 梯度下降的误分类率高于感知器规则
- Perceptron rule (target - thresholded output) guaranteed to converge to a separating hyperplane if the problem is linearly separable. 感知机保证能收敛到一个分离超平面, 如果问题线性可分。

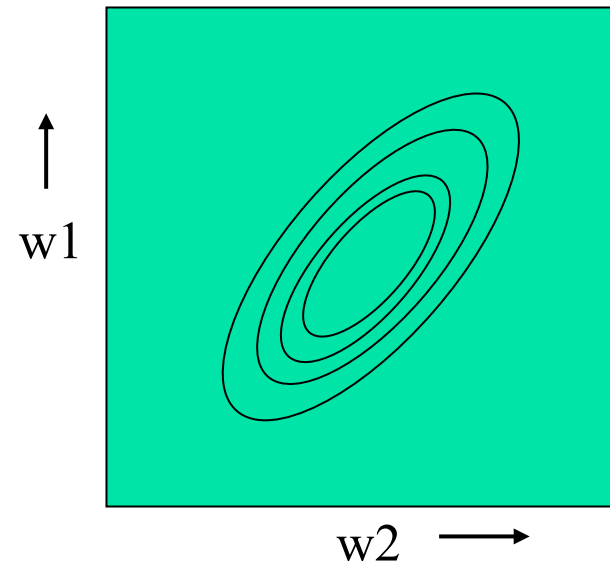
The error surface

- The error surface lies in a space with a horizontal axis for each weight and one vertical axis for the error.
垂直切面是碗型(抛物线), 当只观察误差和一个权重的关系时
- For a linear neuron, it is a quadratic bowl.
- Vertical cross-sections are parabolas.
水平切面是椭圆, 每个椭圆都代表相同误差值的点的集合
- Horizontal cross-sections are ellipses.
中心就是误差最低点

Vertical cross-section



Horizontal cross-section

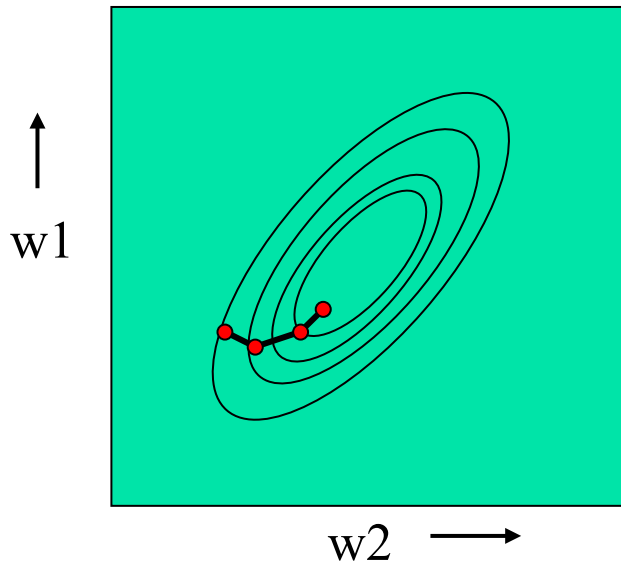


Batch vs incremental learning

- Batch learning does steepest descent on the error surface

在每次更新前都计算了所有的“平均”梯度

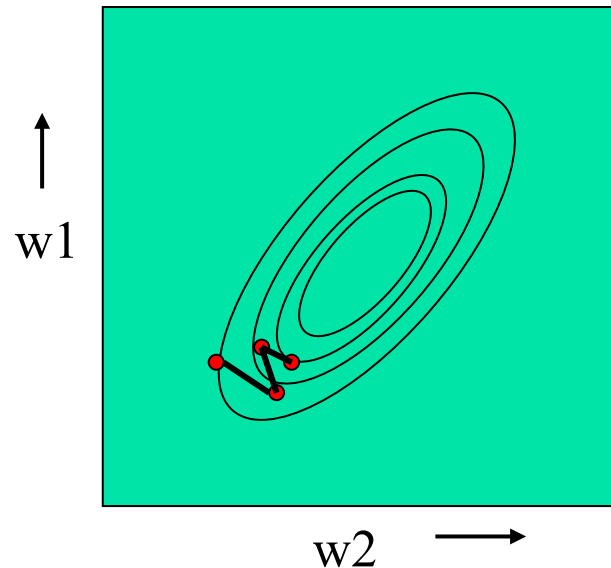
所以下降路径非常平滑

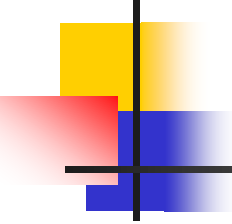


- Incremental learning zig-zags around the direction of steepest descent

因为每次只根据单个样本进行更新

所以它的路径是围绕最速下降方向之字型下降，但整体趋势仍是下降。





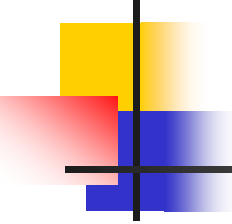
Summary

◆ Perceptron training:

- uses thresholded unit 使用阈值单元
- converges after a finite number of iterations 有限次迭代后收敛
- output hypothesis classifies training data perfectly 完美分类训练数据
- linear separability necessary 线性可分必要性

◆ Gradient descent:

- uses unthresholded linear unit 无阈值的线性单元
- converges asymptotically toward a minimum error hypothesis 渐近地收敛于最小误差解
- termination is not guaranteed 不保证终止
- linear separability not necessary 不必要线性可分



The fall of the Perceptron

- Researchers begun to discover the Perceptron's limitations.
- Unless input categories were "linearly separable", a perceptron could not learn to discriminate between them.
- Unfortunately, it appeared that many important categories were not linearly separable.
- E.g., those inputs to an XOR gate that give an output of 1 (namely 10 & 01) are not linearly separable from those that do not (00 & 11).



THANK YOU



VISIT US

WWW.XJTLU.EDU.CN



FOLLOW US

@XJTLU



Xi'an Jiaotong-Liverpool University

西交利物浦大學